Lab Sheet: Exception Handling in Java

Course: Introduction to Programming Language II (Java)
Topic: Exception Handling, try-catch-finally, throw, throws, Custom Exceptions
Level: Beginner to Intermediate
Duration: 3 hours

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU

1. Objectives

By the end of this lab, students will be able to:

- Understand what exceptions are and why they occur in Java.
- Differentiate between Checked and Unchecked Exceptions.
- Handle exceptions using `try-catch-finally` blocks.
- Use `throw` and `throws` keywords appropriately.
- Implement multiple catch blocks and nested try-catch.
- Create and use User-Defined (Custom) Exceptions.
- Understand exception propagation in method call stack.

2. Introduction to Exception Handling

An **exception** is an abnormal condition that arises during the runtime of a program and disrupts the normal flow of instructions.

Java provides a robust mechanism to handle runtime errors through **Exception Handling**.

Key Keywords: `try`, `catch`, `finally`, `throw`, `throws`

3. Types of Exceptions

- **Checked Exceptions**: Checked at compile-time (e.g., `IOException`).
- **Unchecked Exceptions**: Subclasses of `RuntimeException` (e.g., `ArithmeticException`).
- **Error**: Serious system issues (e.g., `OutOfMemoryError`).

4. Example 1: Basic try-catch

```java
public class BasicExceptionDemo {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a / b;  // ArithmeticException
            System.out.println(result);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        } finally {
            System.out.println("Finally block always executes.");
        }
        System.out.println("Program continues...");
    }
}
```

Output:

```
Exception caught: / by zero
Finally block always executes.
Program continues...
```

5. Example 2: Multiple Catch Blocks

```java
import java.util.*;

public class MultipleCatchDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            int a = sc.nextInt();
            int b = Integer.parseInt(sc.nextLine().trim());
            System.out.println(a / b);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero!");
        } catch (InputMismatchException | NumberFormatException e) {
            System.out.println("Invalid input! Please enter numbers.");
        } finally {
            sc.close();
        }
    }
}
```

6. Example 3: Method Stack & Exception Propagation

```
public class StackDemo {
    public static void main(String[] args) {
        try {
            method1();
        } catch (Exception e) {
            System.out.println("Caught in main: " + e.getMessage());
            e.printStackTrace();
        }
    }

    static void method1() throws Exception {
        method2();
    }

    static void method2() throws Exception {
        throw new Exception("Error occurred in method2");
    }
}
```

7. Example 4: User-Defined Exception

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

class Person {
    void setAge(int age) throws InvalidAgeException {
        if (age < 0) {
            throw new InvalidAgeException("Age cannot be negative!");
        }
        System.out.println("Age set to: " + age);
    }
}

public class CustomExceptionDemo {
    public static void main(String[] args) {
        Person p = new Person();
        try {
            p.setAge(-5);
        } catch (InvalidAgeException e) {
            System.out.println("Custom Exception: " + e.getMessage());
        }
    }
}
```

Tasks & Solutions

Task 1: Basic Exception Handling (15 min)

a. Write a program that takes two integers as input and divides them.

b. Handle `ArithmeticException` and `InputMismatchException`.

c. Use a `finally` block.

Solution:

```
import java.util.*;

public class DivisionTask {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();
            System.out.println("Result: " + (a / b));
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
        } catch (InputMismatchException e) {
            System.out.println("Error: Please enter valid integers!");
        } finally {
            System.out.println("Operation completed.");
            sc.close();
        }
    }
}
```

Task 2: Multiple Exceptions & Nested Try (20 min)

a. Create a program that reads from a file.

b. Handle `FileNotFoundException` and `IOException`.

c. Use nested try-catch for array operations.

Solution:

```
import java.io.*;

public class FileReadTask {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("test.txt");
            try {
                int[] arr = new int[5];
                arr[10] = 100;  // ArrayIndexOutOfBoundsException
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Array Error: " + e.getMessage());
            }
            fr.close();
        } catch (FileNotFoundException e) {
            System.out.println("File not found!");
        } catch (IOException e) {
            System.out.println("IO Error: " + e.getMessage());
        }
    }
}
```

Task 3: Method Overriding with Exceptions (20 min)

a. Create `BankAccount` with `withdraw()` that throws custom exception.

b. Create `SavingsAccount` that overrides with stricter rules.

Solution:

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) {
        super(msg);
    }
}

class BankAccount {
    double balance;
    BankAccount(double bal) { this.balance = bal; }

    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance!")
                ↪ ;
        }
        balance -= amount;
        System.out.println("Withdrawn: " + amount);
    }
}

class SavingsAccount extends BankAccount {
    SavingsAccount(double bal) { super(bal); }

    @Override
    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance * 0.9) {  // Can only withdraw 90%
            throw new InsufficientBalanceException("Cannot withdraw more
                ↪ than 90% from Savings!");
        }
        super.withdraw(amount);
    }
}

public class BankTask {
    public static void main(String[] args) {
        BankAccount acc = new SavingsAccount(10000);
        try {
            acc.withdraw(9500);
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Task 4: Comprehensive Exercise (30 min)

a. Build a simple calculator supporting +, -, *, /.

b. Handle all exceptions gracefully.

c. Add custom exception for invalid operators.

Solution:

```
import java.util.*;

class InvalidOperatorException extends Exception {
    public InvalidOperatorException(String msg) { super(msg); }
}

public class Calculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while (true) {
            try {
                System.out.print("Enter first number: ");
                double a = sc.nextDouble();
                System.out.print("Enter operator (+,-,*,/): ");
                char op = sc.next().charAt(0);
                System.out.print("Enter second number: ");
                double b = sc.nextDouble();

                double result = switch (op) {
                    case '+' -> a + b;
                    case '-' -> a - b;
                    case '*' -> a * b;
                    case '/' -> {
                        if (b == 0) throw new ArithmeticException("Division
                            ↪ by zero");
                        yield a / b;
                    }
                    default -> throw new InvalidOperatorException("Invalid
                        ↪ operator: " + op);
                };
                System.out.println("Result = " + result);
            } catch (Exception e) {
                System.out.println("Error: " + e.getMessage());
            } finally {
                System.out.print("Continue? (y/n): ");
                if (sc.next().toLowerCase().charAt(0) != 'y') break;
            }
        }
        sc.close();
    }
}
```

Keep Coding!

Mastering exception handling is crucial for writing robust, production-ready Java applications.